XIAMEN UNIVERSITY MALAYSIA

Course Code	:	DMT321
Course Name	:	Mobile Application System Design
Lecturer	:	Nurul Amera binti Muhamad Nazmi
Academic Session	:	2026/04
Assessment Title	:	Assignment 2
Submission Due Date	:	26th June 2026

Prepared By :

Student ID	Student Name
DMT2309279	Antonius Michael
DMT2309281	Gunawan Nelsen Keane
DMT2309287	Tando Jocelyne Audrey
DMT2309289	Wilyman Howard

Date Received :

Feedback from Lecturer:

Mark:

XIAMEN UNIVERSITY MALAYSIA

Own Work Declaration

I/We hereby understand my/our work would be checked for plagiarism or other misconduct, and the softcopy would be saved for future comparison(s).

I/We hereby confirm that all the references or sources of citations have been correctly listed or presented and I/we clearly understand the serious consequence caused by any intentional or unintentional misconduct.

This work is not made on any work of other students (past or present), and it has not been submitted to any other courses or institutions before.

Signature: 

Date: 26th June 2026

Table of Contents

1. Introduction 4
2. UI Design 4
   A. Registration Screen 4
   B. Log in Screen 5
   C. Home Screen 5
   D. Profile Screen 6
   E. Settings Screen 6
   F. Content/Feature Screen 7
   G. Help/Support Screen 9
3. CRUD (Firebase) 9
   A. Change Username 9
   B. Add Friend 11
   C. Accept Friend Request 13
   D. Friends List 15
   E. Delete Friend 16
4. Reflection 18

1. Introduction

For this project, we developed a mobile application called ‘CODU’, a gamified beginner friendly educational application which aims to teach programming for new learners. Inspired from other gamified learning application, we created a friendly and welcoming environment by using colourful elements and building a game-inspired user interface. ‘CODU’ offers courses of multiple programming languages such as Python, C++, JavaScript and Java. These lessons are designed to be simple to understand, with its interactive questions, and is structured to progress in difficulty fairly, by providing more challenge in each level and an assessment in every 5 levels that requires users to write code form scratch. By designing the levels using this structure, it provides users with fun and interactive experience while allowing them to demonstrate coding and problem-solving skills.

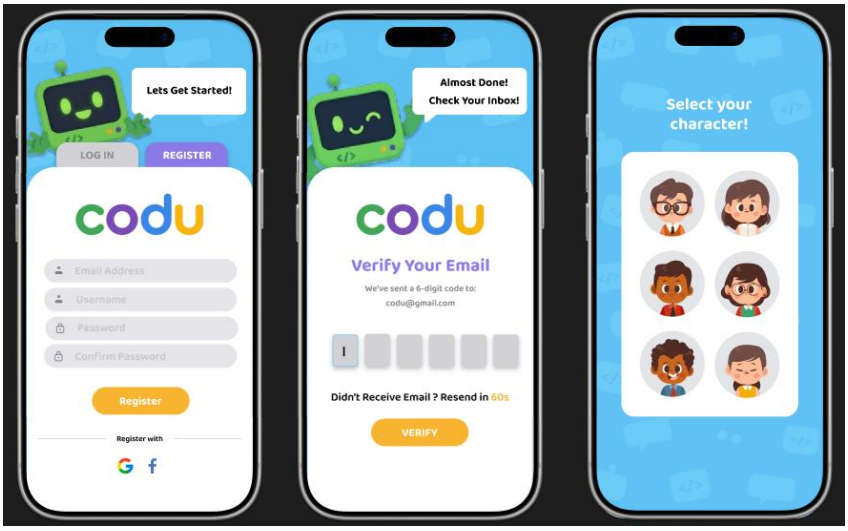
Additionally, other features in ‘CODU’ includes a ‘Duel Mode’, allowing users to challenge their friend or other players in real-time programming exercises. By implementing a point system based on the user speed and accuracy, and with rewards such as trophies or ranks points, ’CODU’ creates a competitive environment for players to test the skills they learned. In addition, the ‘Duel Mode’ is further supported by global leaderboards and achievement badges for enhanced enjoyability.

Overall, ‘CODU’ combines programming education, gamification, and social competition to create an enjoyable and interactive learning experience, helping beginners of all ages develop fundamental coding skills.

2. UI Design

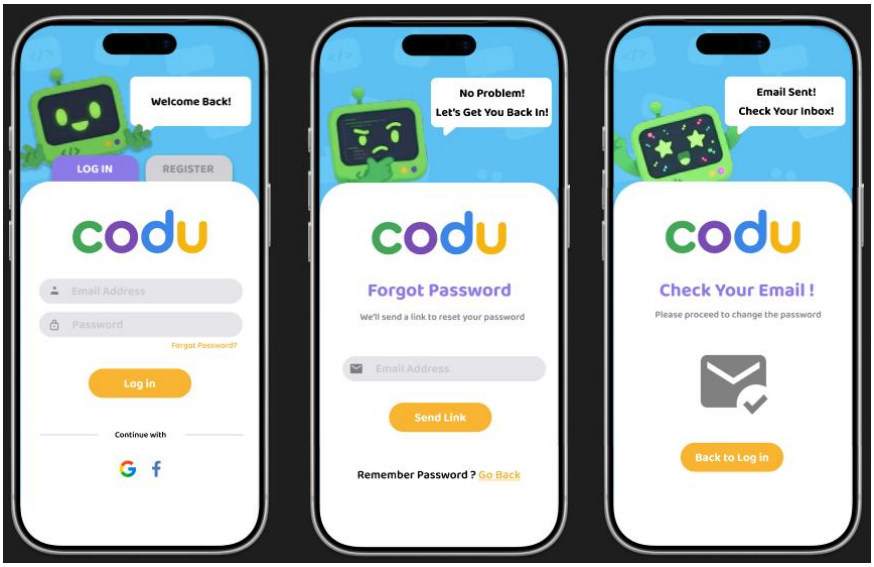
A. Registration Screen

On the registration page, user need to input their username, email address, and their password, or register using Google or Facebook account. Clicking the register button will ask the user to verify their email using code. After that, user select one of the characters available for their profile.



B. Log in Screen

On the log in page, user can input their email address and password or using their Google or Facebook account to log in. When user forgets their password, user can reset it by clicking "Forgot Password" and input their email, and the app will send a link to the email to proceed.



C. Home Screen

After register/login user is directed to the home page, showing the list of subjects and their history to view their learning progress. Streak point system is also available where user can get 1 streak point each day by completing a level, and the streak will reset to 0 when user misses a day. Clicking the subject will lead to the subject level UI that shows their current progress and the stars earned.



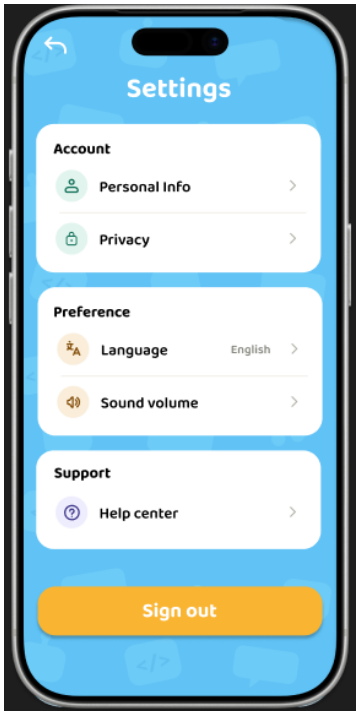
D. Profile Screen

On the profile page, user can see and edit their username and characters. It shows the user earned streaks, trophies, and badges. Got “Friend List” button that will show the user's friends and can challenge them for the quiz duel, as well as an " Add " button to add a new friend. Got a settings button at the upper right that leads to the settings UI.



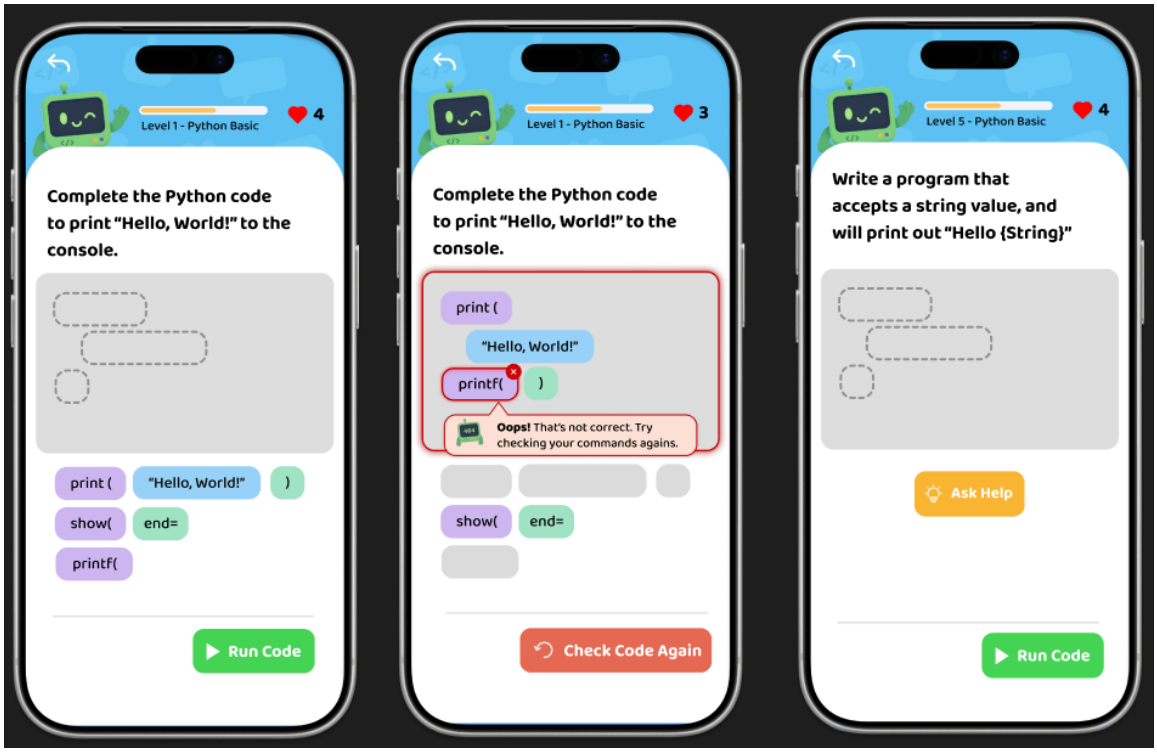
E. Settings Screen

On the settings page, user can see and change account settings of their personal info and privacy. The user can also change the app language and adjust the app sound volume. For the support center, user can click on the help center button that will direct to the help center UI.

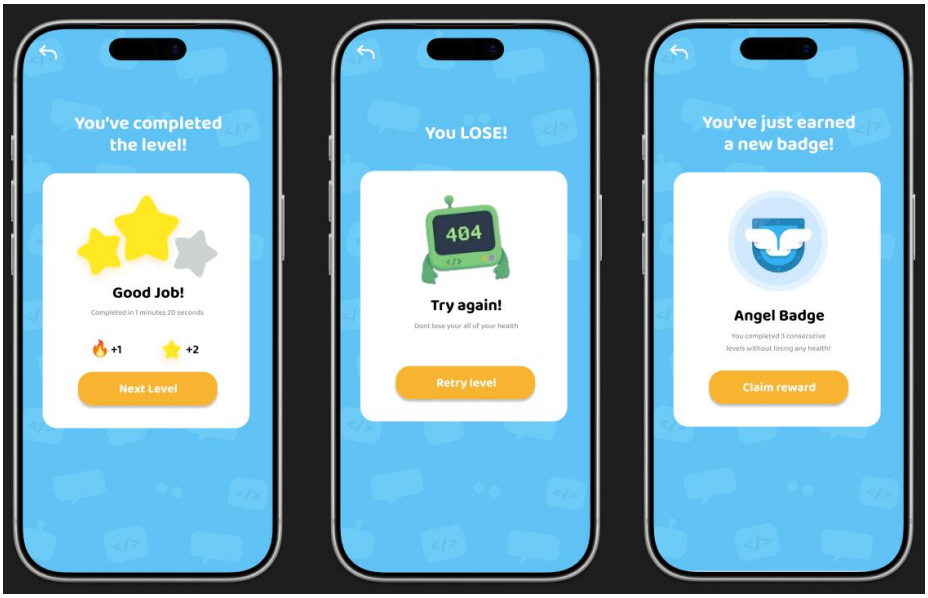


F. Content/Feature Screen

The main feature of the app is the learning UI, where user need to answer questions about the related programming language. User need to answer the questions correctly with a health remaining to pass the level. Wrong answers result in losing 1 heart.



When user complete the level they will get 1 streak point that is limited per day and stars based on their performance (full heart = 3 stars, heart>1 = 2 stars, heart=1 = 1 star). User will lose the game when their heart is 0. User can earn a new badge when they complete a hidden mission.

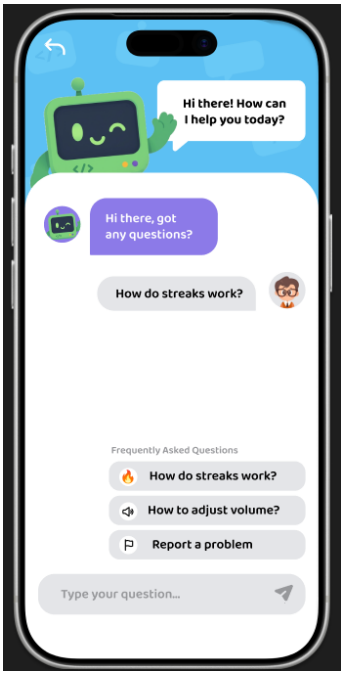


Another feature is the duel system, where user can compete quiz with their friends or random people. User can select the programming language they want, and the app will search for a suitable opponent based on a similar number of trophies, and the winner of the duel will earn trophies. The leaderboard UI shows the top user who earned the most trophies globally and between friends.



G. Help/Support Screen

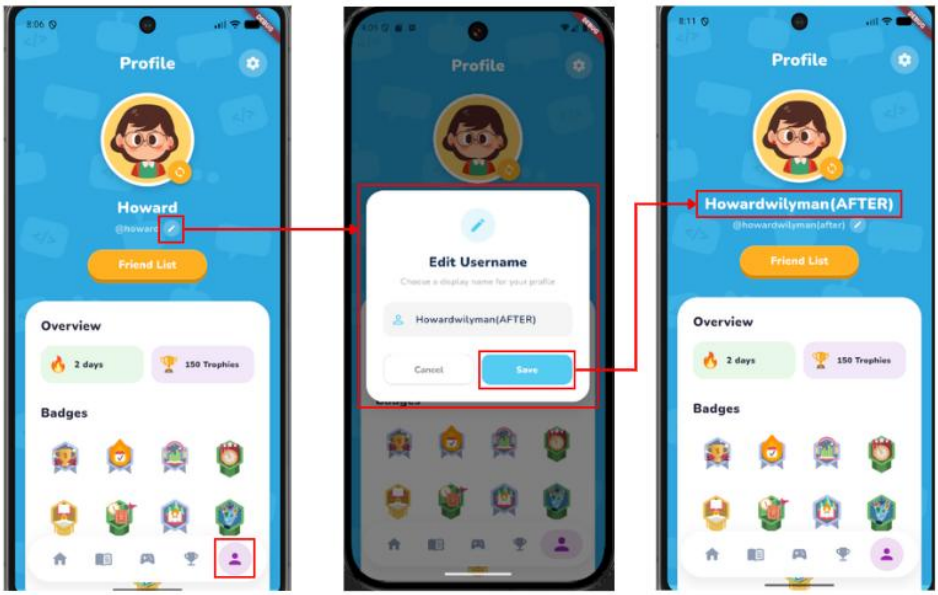
On the help center UI, user can interact with the chatbot to get an answer or receive assistance. User can type their questions or choose from the FAQ list.



3. CRUD (Firebase)

A. Change Username

In CODU, the Change Username function use *Update* to update the user's display name.



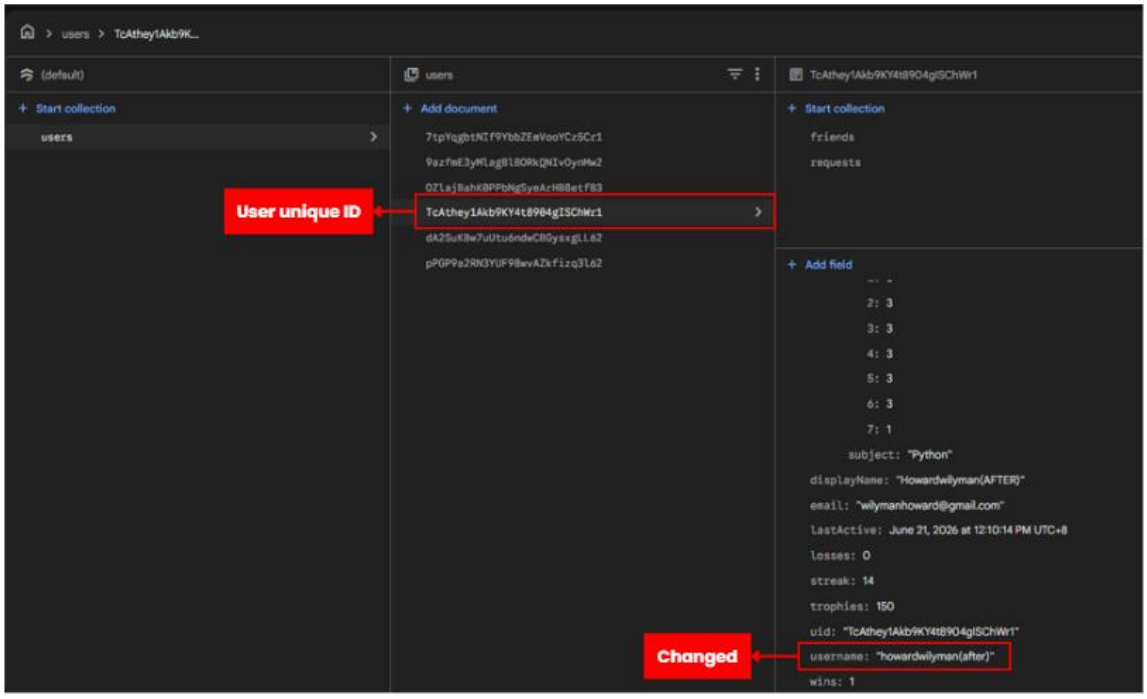
When the user changes their username in the Profile page by typing a new name and tapping the "Save" button, the *saveDisplayName()* function is triggered, updating the name locally on the device and in the Firebase so other devices can see the change.

- **Code Snippets**

```dart
// Save custom display name locally and to Firebase Auth
Future<void> saveDisplayName(String name) async {
  final uid = _uid;
  if (uid == null) return;
  final prefs = await SharedPreferences.getInstance();
  await prefs.setString('display_name_$uid', name);
  // Also push to Firebase so other devices see it
  try {
    await FirebaseAuth.instance.currentUser?.updateDisplayName(name);
  } catch (_) {
    // If Firebase update fails, local storage is still saved
  }
 }
}
```

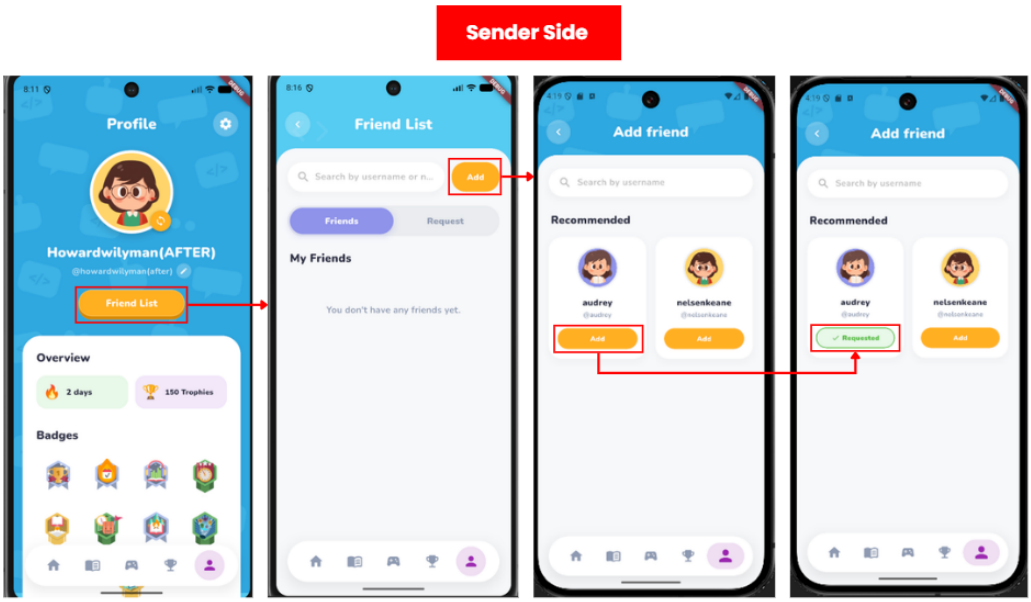
1. Saves the new display name locally on the device using *SharedPreferences* to make sure that the name loads instantly on the user's own device without needing to fetch from Firebase every time.
2. Updates the display name in Firebase Auth so it changes in all devices.
3. If the Firebase update fails, the name is still saved locally.

- **Firebase**

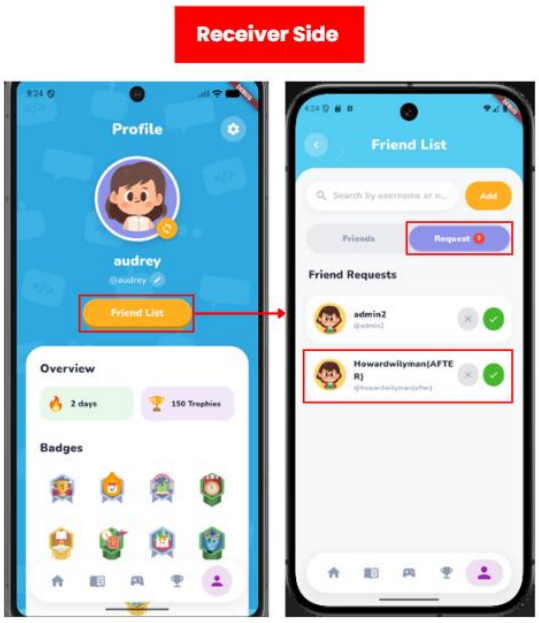


B. Add Friend

In CODU, the Add Friend function use *Insert* to save a friend request into Firestore database. When the user adds a friend by tapping the "Add" button, the *sendFriendRequest()* function is triggered creating 2 new records in the database at the same time, one on the sender side and another on the receiver side.

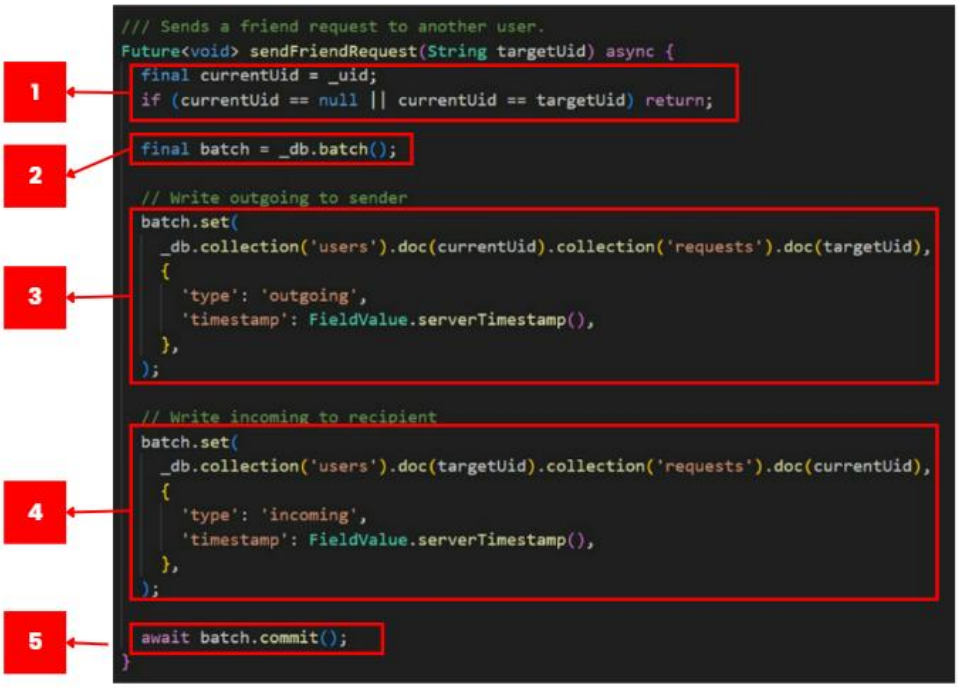


On the sender's side, the "Add" button change to "Requested" to show that the request has been sent.



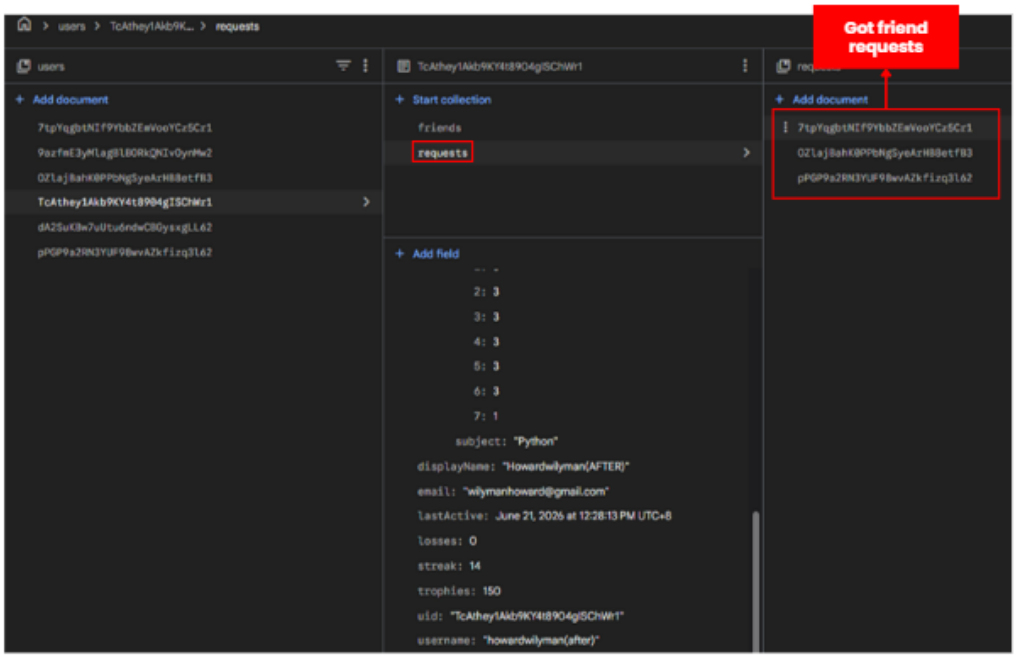
On the receiver's side, the name of the sender will appear in the receiver's "Request" section, notifying that someone send them a friend request.

- **Code Snippets**



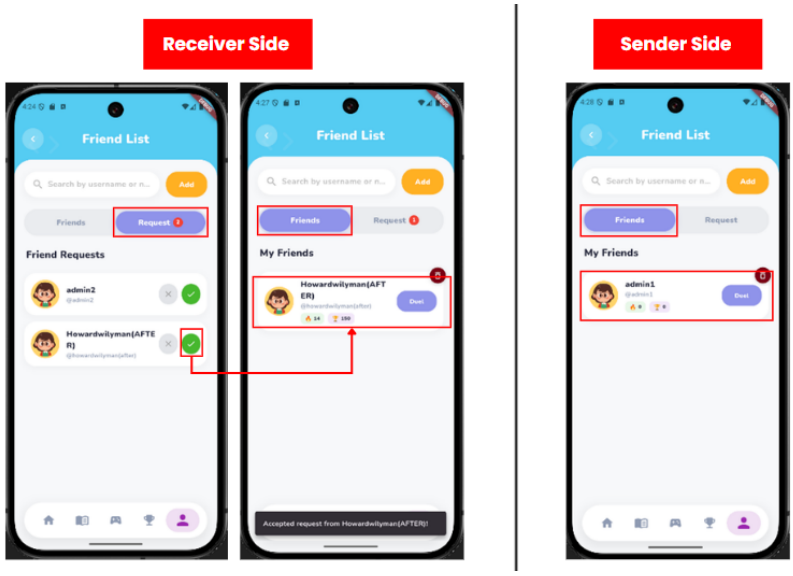
1. Safety check that makes sure the user is logged in and not trying to add themselves. If the condition is not met, the function will be stopped and nothing will be written in the database.
2. Prepare the batch write that making sure both of the insert (step 3 and 4) are executed at the same time, preventing only one side of the request get saved.
3. Insert the outgoing requests to the sender requests subcollection in the Firestore, marked as "outgoing".
4. Insert the incoming request to the receiver requests subcollection in the Firestore, marked as "incoming".
5. Send the 2 insert operations to the Firestore at the same time.

- **Firebase**



C. Accept Friend Request

In CODU, the Accept Friend Request function use *Delete* and *Insert* logic.



When the receiver accepts friend request by tapping the "✓" button, the *acceptFriendRequest()* function is triggered, deleting the request record in the sender and receiver *"Request"* section. At the same time, the name of the sender is added in the receiver

friend list, and the receiver's name will be added in the sender friend list, making them successfully become friends in the app.

- **Code Snippets**

```
Future<void> acceptFriendRequest(String targetUid) async {
  final currentUid = _uid;
  if (currentUid == null) return;

  final batch = _db.batch();

  // Remove requests docs
1 batch.delete(_db.collection('users').doc(currentUid).collection('requests').doc(targetUid));
2 batch.delete(_db.collection('users').doc(targetUid).collection('requests').doc(currentUid));

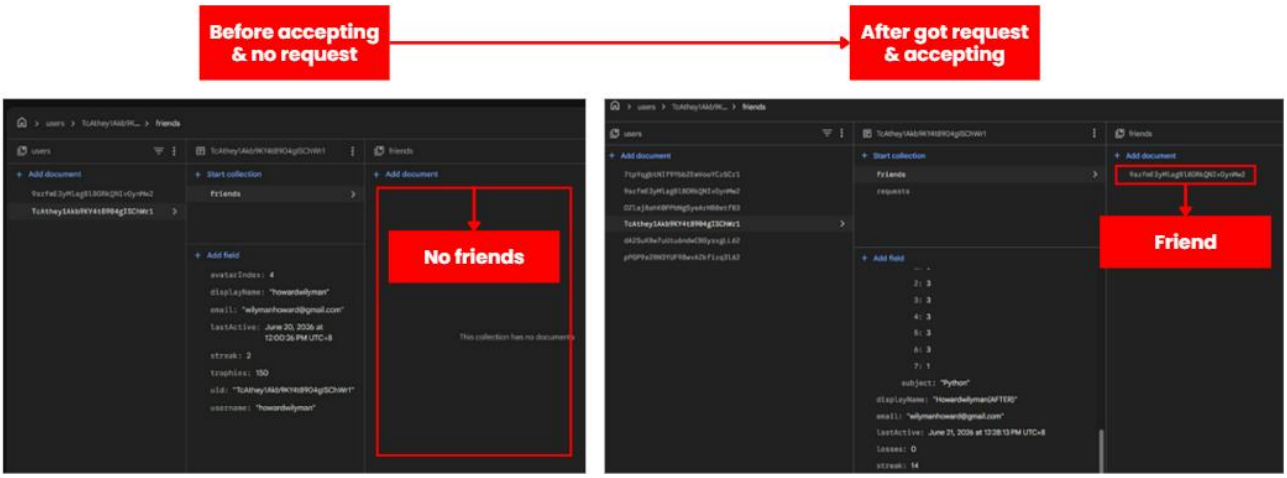
  // Add friend doc to current user
3 batch.set(_db.collection('users').doc(currentUid).collection('friends').doc(targetUid), {
    'timestamp': FieldValue.serverTimestamp(),
  });

  // Add friend doc to target user
4 batch.set(_db.collection('users').doc(targetUid).collection('friends').doc(currentUid), {
    'timestamp': FieldValue.serverTimestamp(),
  });

5 await batch.commit();
}
```

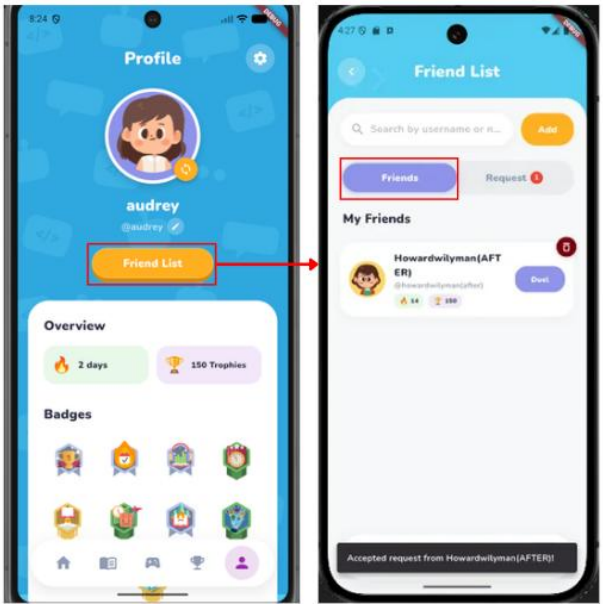
1. Delete the request from the receiver's requests subcollection in Firestore, removing the request from the receiver's Request section.
2. Delete the request from the sender's requests subcollection in Firestore, removing the "Requested" status from the sender's side.
3. Insert the sender details into the receiver's friends subcollection in Firestore, adding the sender's name in the receiver's friend list.
4. Insert the sender details into the receiver's friends subcollection in Firestore, adding the sender's name in the receiver's friend list.
5. Send the 2 delete and 2 insert operations to the Firestore at the same time.

- **Firebase**



D. Friends List

In CODU, the Friends List function use *Retrieve* logic to fetch and display the current user's friend list from the Firestore database.



When use tap the “Friend List” button on the profile UI, the *streamFriends()* function is triggered, showing the user current friend list that is continuously updated in real time.

- **Code Snippets**

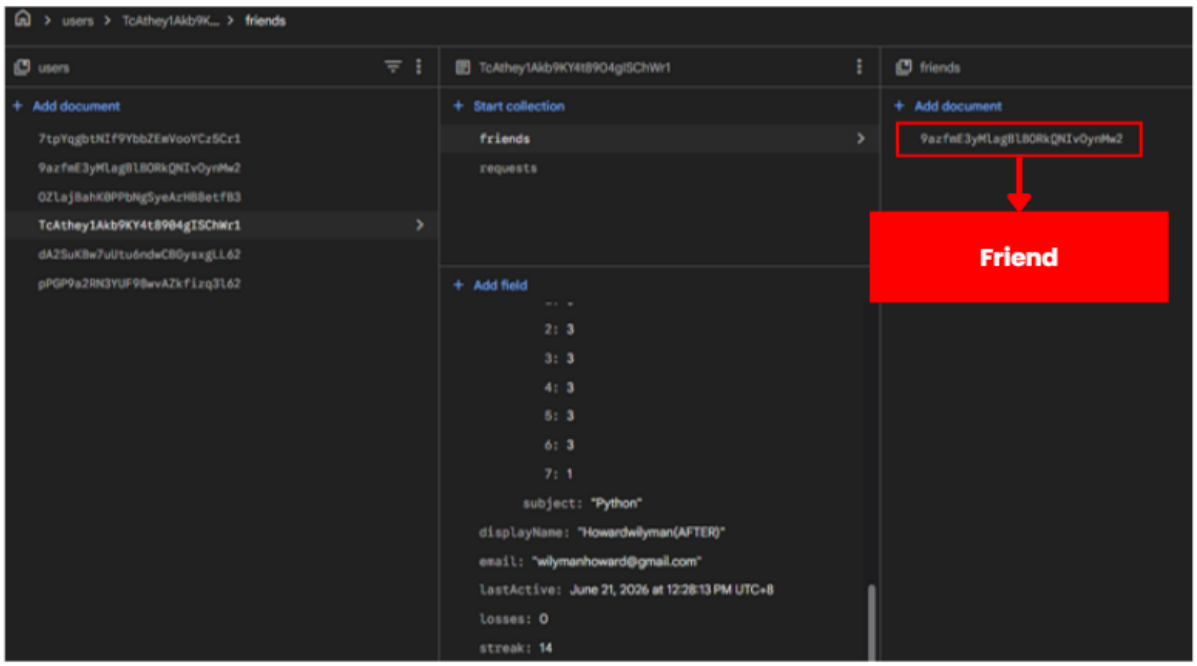
```dart
/// Streams the list of friends for the current user.
Stream<List<Map<String, dynamic>>> streamFriends() {
  final currentUid = _uid;
  if (currentUid == null) return const Stream.empty();

  return _db
      .collection('users')
      .doc(currentUid)
      .collection('friends')
      .snapshots()
      .asyncMap((snapshot) async {
    List<Map<String, dynamic>> friendsList = [];
    for (var doc in snapshot.docs) {
      final friendUid = doc.id;
      // Fetch friend's latest profile details
      final userDoc = await _db.collection('users').doc(friendUid).get();
      if (userDoc.exists) {
        friendsList.add(userDoc.data()!);
      }
    }
    return friendsList;
  });
}
```

1
2
3
4

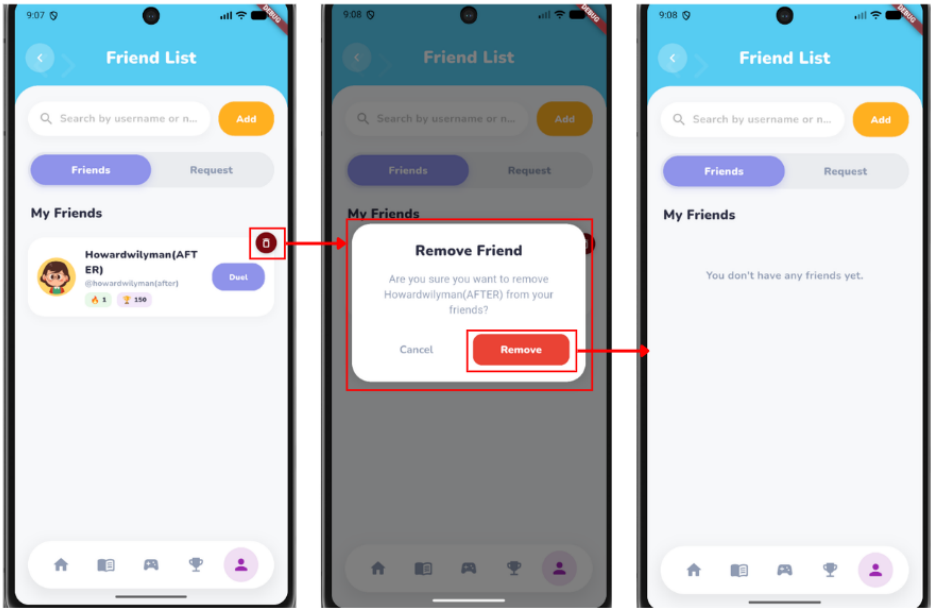
1. Access the current user's friends subcollection in Firestore and listen to it using *snapshots(),* so the friend list will be automatically updated on the screen every change happens in the database, in the case of adding or removing a friend
2. Retrieve the friend's UID from the document ID for each document found in the friends subcollection.
3. Use the friend's UID to fetch their latest profile details from the user's collection in Firestore, displaying their name, avatar, streaks, and trophies.
4. If the profile document exists, it will add the friend's details to the friendsList and be displayed on screen.

- **Firebase**



E. Delete Friend

In CODU, the Remove Friend function use *Delete* logic to remove a friend from the Firestore.



When user remove a friend by tapping the trash bin button on the target user profile, the *removeFriend()* function is triggered, deleting the friend's name in the user friend list and deleting the user's name in the target user's friend list.

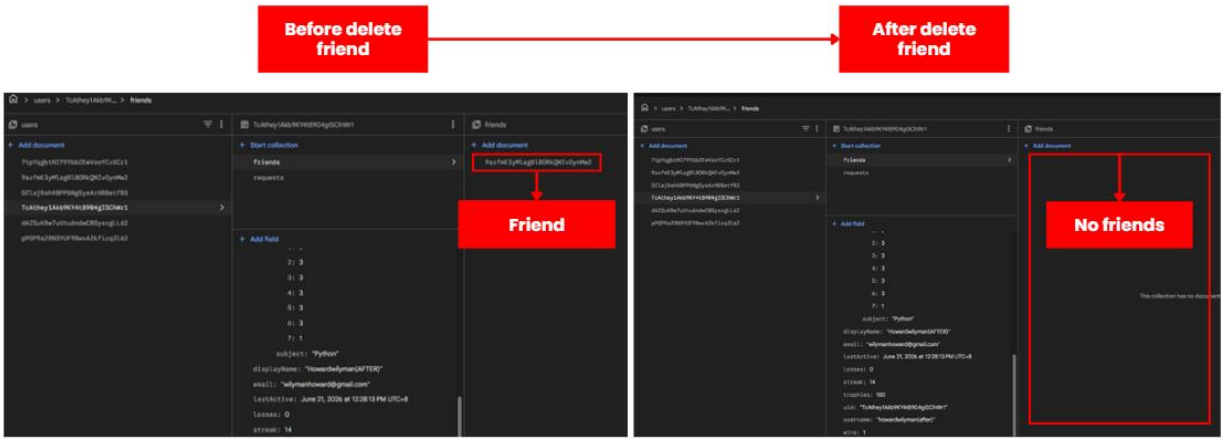
- **Code Snippets**

```
/// Removes a friend.
Future<void> removeFriend(String targetUid) async {
  final currentUid = _uid;
  if (currentUid == null) return;

  final batch = _db.batch();
  batch.delete(_db.collection('users').doc(currentUid).collection('friends').doc(targetUid));
  batch.delete(_db.collection('users').doc(targetUid).collection('friends').doc(currentUid));
  await batch.commit();
}
```

1. Delete the target user from the user's friends subcollection in Firestore, removing the target user from the user friend list.
2. Delete the user from the target user's friends subcollection in Firestore, removing the user from the target user's friend list.
3. Send 2 delete operations to Firestore at the same time.

- **Firebase**



4. Reflection

When we develop the CRUD features for our learning game app CODU, we learn something new, for example applying a real time scalable mobile application using the firebase authentication, Cloud Firestore, and SharedPreferences. By adding real-time friends list in our application, we learn to utilize Firestore snapshots to create a highly responsive user experience, where the database changes instantly update on the UI, meaning that no manual refreshes are needed. In addition, managing the interactions like sending, accepting, and deleting friend requests taught us the importance of data integrity in this sector. Moreover, we use Firestore to guarantee that mutual data operations either succeeded together or fail together. Ensuring the data between users to match. Finally, when we combined local caching via SharedPreferences with cloud updates, we realized that the UI responsiveness can be balanced with backend persistent.

The process in developing this mobile app is not free from challenges, especially when managing more than one user accounts. A major issue that faced it was the risk of “one-sided updates” where an interruption might happen and cause a friend request or deletion to process on sender side but fail on the receiver side. We resolved it by bundling the related operations into a single batch commit, that make sure synchronization between the user accounts. Another major issue is the risk of UI lag when updating user information. To solve this, we designed a hybrid strategy that allow us to change to local device storage while also handling the Firebase update asynchronously in the background.

Finishing our project, we realize few improvement that we can implement in our future updates. In the future, we might consider moving the friend and dual logic in our application to Firebase Cloud Functions, since it would better isolate the database and prevent users from writing data directly to other users' profiles. To make our app more efficient, we would also like to implement query pagination and limits on the leaderboard and the friend list, in order to prevent performance degradation. Finally, we aim to fully enable Firestore's offline persistence feature so that users can still do actions, manage settings, and track their daily streaks even when the network drops.

APPENDIX 1

MARKING RUBRICS

Criteria	Poor (0-2)	Average (3-5)	Good (6-8)	Excellent (9-10)	Poor (0-2)	Weight (%)	Mark
Android App Development & Design Consistency (Team assessment with individual adjustment) (Task 1)	Idea unclear or app poorly structured.	App concept basic with limited design consistency.	Clear concept; design fairly consistent and usable.	Professionally presented UI with consistent layout and strong visual design.	Idea unclear or app poorly structured.	**10**	
	Poor (0-2)	**Average (3-5)**	**Good (6-8)**	**Excellent (9-10)**	**Poor (0-2)**		
App Functionality & Feature Implementation (Team assessment with individual adjustment) (Task 2)	Core feature incomplete or non-functional.	Basic feature partially working; limited interactivity.	Functional feature (CRUD, map, or media) with minor issues.	Fully working feature with smooth navigation and proper logic.	Core feature incomplete or non-functional.	**10**	
	Poor (0-2)	**Average (3-5)**	**Good (6-8)**	**Excellent (9-10)**	**Poor (0-2)**		
Report Explanation & Reflection (Team assessment with individual adjustment) (Task 3)	written; missing explanation or reflection.	Limited explanation and weak reflection.	Clear explanation with basic reflection on learning.	Well-written report with strong technical explanation and insightful reflection.	written; missing explanation or reflection.	**10**	

TOTAL	**/30**	
Deduction for late submission or failure to adhere to the submission format.	**0 - 10**	
Final Team Mark	**/30**	

Individual Mark Adjustments for Contribution, Performance, and AI-Generated Content

Student Name: Student ID:	Fill in your ID and Name here	Fill in your ID and Name here	Fill in your ID and Name here	Fill in your ID and Name here	Fill in your ID and Name here
Team Mark					
Mark Deduction					
Mark Addition					
Final Individual Mark					

Members' Contribution Sheet

Team Member Name & ID	Tasks and writing	Signature
Antonius Michael DMT2309279	• UI (Log in, Register, Email Verification, Forgot Password, Email Sent, Duel Selection, Duel Win) • Writing: Reflection	
Gunawan Nelsen Keane DMT2309281	• UI (Home page, Exam Question, Profile, Friend List) • UI navigation implementation • Writing: Introduction	
Tando Jocelyne Audrey DMT2309287	• UI (Character Selection, Level Path, Win Level, Lose Level, Earn Badge, Duel Match Found, Duel Battle, Leaderboard, Add Friend, Help Center) • Writing: UI Design, CRUD	
Wilyman Howard DMT2309289	• UI (Question, Wrong Question, Settings, Add Friend) • Firebase setup and implementation	

Note to students: Please include the marking rubric when submitting your coursework.

3% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Match Groups

- **7** Not Cited or Quoted 3%
  Matches with neither in-text citation nor quotation marks
- **0** Missing Quotations 0%
  Matches that are still very similar to source material
- **0** Missing Citation 0%
  Matches that have quotation marks, but no in-text citation
- **0** Cited and Quoted 0%
  Matches with in-text citation present, but no quotation marks

Top Sources

0% Internet sources
2% Publications
2% Submitted works (Student Papers)